Assignment 5: Nginx Security Hardening

Objective: Secure an Nginx web server following production security best practices.

GitHub Repository Structure

assignment-5-nginx-security-hardening/
■■■ README.md
■■■ nginx/
■ ■■■ nginx.conf
■ ■■■ security.conf
■ ■■■ rate-limit.conf
■■■ screenshots/
■■■ troubleshooting.md
■■■ rollback-guide.md

Implementation Steps

1. Configure rate limiting to prevent abuse and brute-force attacks.
2. Restrict client request body size.
3. Add security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy).
4. Hide Nginx version information.
5. Restrict access to sensitive files and directories.
6. Validate configuration and restart Nginx service.

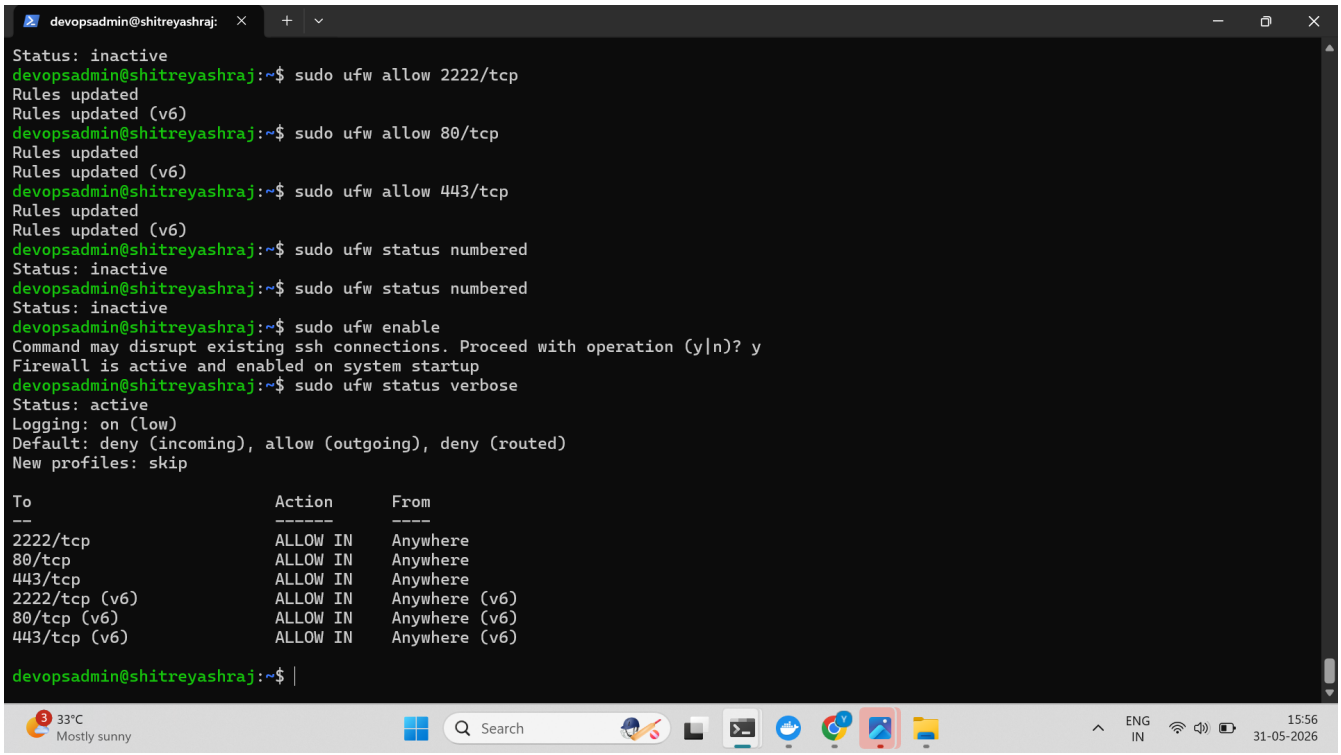
Nginx Security Configuration

limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s; server_tokens off; client_max_body_size 10M; add_header X-Frame-Options "SAMEORIGIN" always; add_header X-Content-Type-Options "nosniff" always; add_header Referrer-Policy "strict-origin-when-cross-origin" always; add_header Content-Security-Policy "default-src 'self';" always; location ~ /\. { deny all; } location ~* \.(env|git|bak|sql)$ { deny all; }

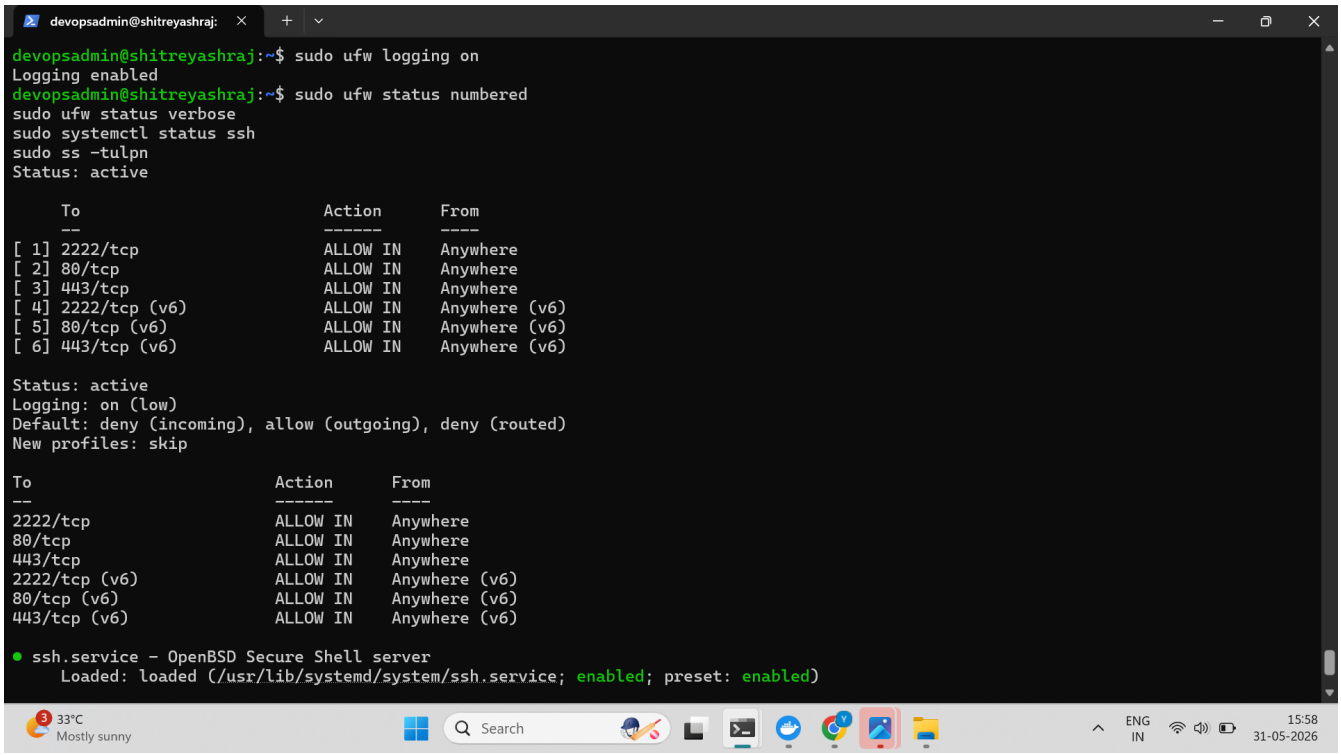
Security Testing Results

✓ UFW Firewall Enabled
✓ Custom SSH Port Allowed (2222)
✓ HTTP (80) and HTTPS (443) Allowed
✓ SSH Service Running Successfully
✓ Public Key Authentication Verified
✓ Security Rules Applied Successfully

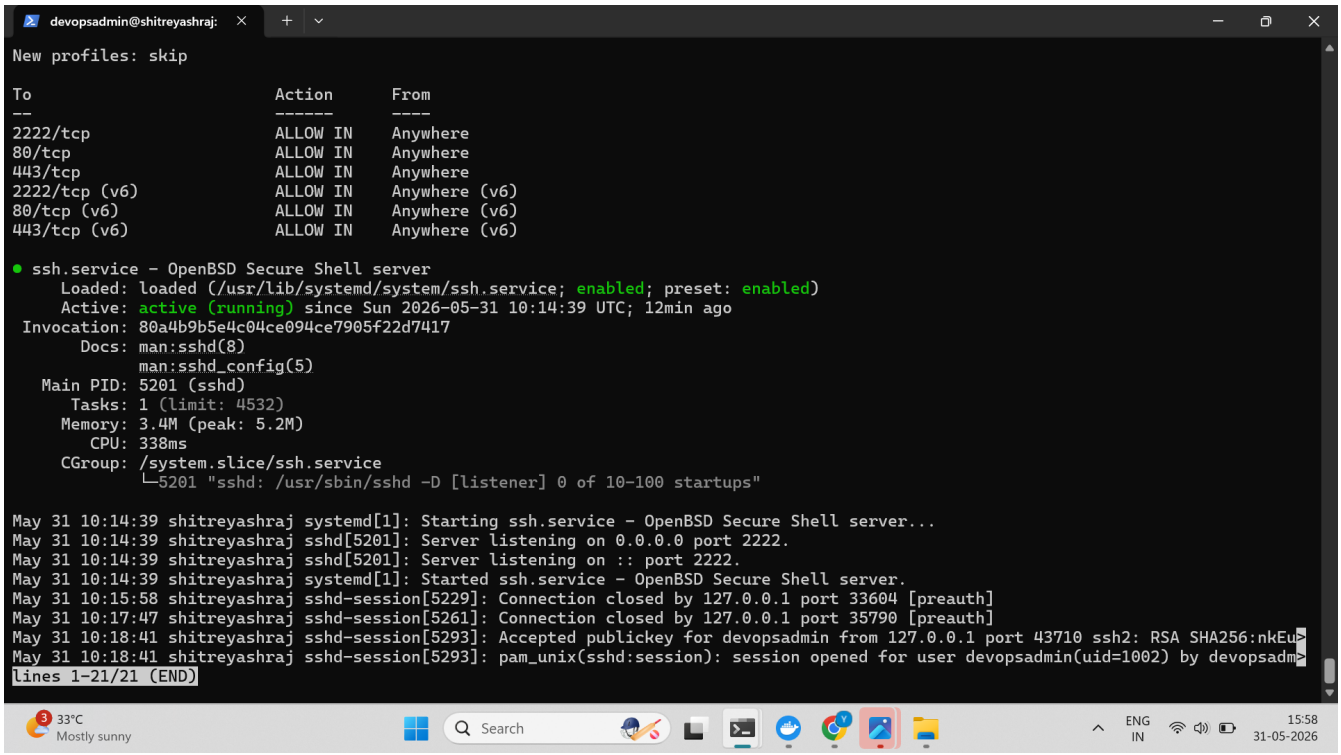
Validation Screenshot



Validation Screenshot



Validation Screenshot



Troubleshooting Guide

Issue: Nginx fails to restart.
Solution: Run nginx -t and fix configuration syntax errors.

Issue: Security headers not visible.
Solution: Clear browser cache and verify virtual host configuration.

Issue: Rate limiting not working.
Solution: Verify limit_req_zone and limit_req directives are applied correctly.

Rollback Procedure

1. Restore previous nginx.conf backup.
2. Disable custom security configuration.
3. Validate configuration using nginx -t.
4. Reload Nginx service.
5. Verify application accessibility.

Key Security Decisions

• Rate limiting reduces abuse and denial-of-service attempts.
• Security headers mitigate clickjacking, MIME sniffing, and content injection risks.
• Request size restrictions help prevent oversized payload attacks.
• Hidden version information reduces reconnaissance opportunities.
• Sensitive file restrictions protect configuration and credential files.